Churn Prediction Modeling

LEPRINCE DONGMO NANDA

2026-10-13

Table of contents

1 Pipeline overview (business view) 8

2 Understanding the results 10
2.1 Logistic Regression . . . 10
2.2 Random Forest . . . 11
2.3 Which model is better? . . . 11
2.4 Why is churn hard to predict here? . . . 11

3 1. Individual probabilities (customer-level view) 11

4 2. Segment probabilities into risk bands 12

5 3. Distribution of the predicted probabilities 12

6 4. Example customers by probability level 13

7 5. Share of customers in each risk band 13

8 6. Export the customer churn probabilities (CSV / Excel) 14

9 Scoring new customers with the saved Random Forest pipeline 14

10 Save the selected model (Random Forest pipeline) 14

11 Conclusion 15

Introduction & Business Context

Customer churn—defined as the rate at which clients end their relationship with a company—is a critical performance indicator across subscription-based and service-driven industries. In the banking sector in particular, churn leads not only to lost revenue but also significant acquisition costs, as winning a new customer often requires 5 to 10 times more investment than retaining an existing one. For a multinational bank, maintaining long-term customer relationships is therefore essential for ensuring financial stability, sustaining growth, and maximizing customer lifetime value (CLV).

This project is built using an anonymized Multinational Bank Customer dataset obtained from Kaggle, containing demographic details, customer behavior, product usage, and account status information. The goal is to develop a predictive machine learning model that estimates the probability of customer churn, enabling the organization to:

Detect at-risk customers early

Prioritize retention strategies and allocate marketing budgets effectively

Personalize offers and services

Strengthen customer loyalty and improve CLV

Support evidence-based, data-driven decision-making

Project Scope & End-to-End Workflow

This end-to-end data science and business analytics project covers the complete pipeline from raw data to actionable insights:

1. Data Acquisition

Dataset sourced from Kaggle (Bank Churn / Customer Churn dataset).

Imported into Jupyter Notebook for preprocessing and modeling.

2. Exploratory Data Analysis (EDA)

Statistical description and distribution analysis

Correlation scan of every feature against the churn target

Identification of data quality issues (missing values, target leakage, class imbalance)

3. Data Preprocessing

Checking for missing values (none found) and removing identifier and leakage columns

Encoding categorical features

Scaling numerical variables

Measuring class imbalance and choosing evaluation metrics that account for it (precision, recall, F1 on the minority class)

Splitting into stratified training and test sets (80 / 20)

4. Machine Learning Modeling

Training two models (Logistic Regression, Random Forest)

Model comparison on a stratified hold-out set

Model evaluation using accuracy, precision, recall, F1-score and ROC-AUC

Feature importance analysis to identify key churn drivers

5. Prediction & Deployment Preparation

Generating customer-level churn probability scores

Exporting results for operational teams

Preparing a clean dataset for BI dashboards

6. Business Intelligence & Visualization (Power BI)

A one-page Power BI report that consumes a churn score per customer:

Headline KPIs (customers, accounts, transactions, average churn probability, customers at risk)

Value and risk segmentation of the customer base

Churn exposure by relationship manager and by city

Note: the Power BI report runs on synthetic relational tables (customers, accounts, transactions, products). Its churn score column is simulated; it shows how a model score is consumed in a decision-support layer.

```
#  SECTION 1 - Import Required Libraries
# Pandas is used to load and manipulate data tables
import pandas as pd

# Splits data into training and test samples
from sklearn.model_selection import train_test_split

# Preprocessing tools:
# - OneHotEncoder: converts categorical labels into numerical columns
# - StandardScaler: normalizes numerical values
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# ColumnTransformer: lets us apply different preprocessing steps to different column types
from sklearn.compose import ColumnTransformer

# Pipeline: chains preprocessing steps + model into one unified workflow
from sklearn.pipeline import Pipeline

# Classification performance metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Machine learning algorithms
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

```
#  SECTION 2 - Load Dataset
# Loads the CSV file into a pandas DataFrame.
# The file sits next to this notebook (Kaggle - Bank Customer Churn dataset).
df = pd.read_csv("Customer-Churn-Records.csv")
```

```
df.shape
```

```
(10000, 18)
```

```
import pandas as pd


df.head(10).T
```

	0	1	2	3	4	5	6	7	8	9
RowNumber	1	2	3	4	5	6	7	8	9	10
CustomerId	15634602	15647311	15619304	15701354	15737888	15574012	15592531	15656148	15792365	15592389
Surname	Hargrave	Hill	Onio	Boni	Mitchell	Chu	Bartlett	Obinna	He	H?
CreditScore	619	608	502	699	850	645	822	376	501	684
Geography	France	Spain	France	France	Spain	Spain	France	Germany	France	France
Gender	Female	Female	Female	Female	Female	Male	Male	Female	Male	Male
Age	42	41	42	39	43	44	50	29	44	27
Tenure	2	1	8	1	2	8	7	4	4	2
Balance	0.0	83807.86	159660.8	0.0	125510.82	113755.78	0.0	115046.74	142051.07	134603.88
NumOfProducts	1	1	3	2	1	2	2	4	2	1
HasCrCard	1	0	1	0	1	1	1	1	0	1
IsActiveMember	1	1	0	0	1	0	1	0	1	1
EstimatedSalary	101348.88	112542.58	113931.57	93826.63	79084.1	149756.71	10062.8	119346.88	74940.5	71725.73
churn	1	0	1	0	0	1	0	1	0	0
Complain	1	1	1	0	0	1	0	1	0	0
Satisfaction Score	2	3	3	5	5	5	2	2	3	3
Card Type	DIAMOND	DIAMOND	DIAMOND	GOLD	GOLD	DIAMOND	SILVER	DIAMOND	GOLD	GOLD
Point Earned	464	456	377	350	425	484	206	282	251	342

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 18 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   RowNumber           10000 non-null  int64  
 1   CustomerId          10000 non-null  int64  
 2   Surname             10000 non-null  object 
 3   CreditScore         10000 non-null  int64  
 4   Geography           10000 non-null  object 
 5   Gender              10000 non-null  object 
 6   Age                 10000 non-null  int64  
 7   Tenure              10000 non-null  int64  
 8   Balance             10000 non-null  float64
 9   NumOfProducts       10000 non-null  int64  
 10  HasCrCard           10000 non-null  int64  
 11  IsActiveMember      10000 non-null  int64  
 12  EstimatedSalary     10000 non-null  float64
 13  churn               10000 non-null  int64  
 14  Complain            10000 non-null  int64  
 15  Satisfaction Score  10000 non-null  int64  
 16  Card Type           10000 non-null  object 
 17  Point Earned        10000 non-null  int64  
dtypes: float64(2), int64(12), object(4)
memory usage: 1.4+ MB
```

```
##identifying how much data is missing
df.isnull().sum()
```

```
RowNumber           0
CustomerId          0
Surname             0
CreditScore         0
Geography           0
Gender              0
Age                 0
Tenure              0
Balance             0
NumOfProducts       0
HasCrCard           0
IsActiveMember      0
```

```
EstimatedSalary       0
churn                 0
Complain              0
Satisfaction Score    0
Card Type             0
Point Earned          0
dtype: int64
```

```
## if we want to see the result in percentage
df.isnull().mean().sort_values(ascending=True)
```

```
RowNumber             0.0
CustomerId            0.0
Surname               0.0
CreditScore           0.0
Geography             0.0
Gender                0.0
Age                   0.0
Tenure                0.0
Balance               0.0
NumOfProducts         0.0
HasCrCard             0.0
IsActiveMember        0.0
EstimatedSalary       0.0
churn                 0.0
Complain              0.0
Satisfaction Score    0.0
Card Type             0.0
Point Earned          0.0
dtype: float64
```

```
import pandas as pd

df.describe().T.head(10)
```

	count	mean	std	min	25%	50%	75%	max
RowNumber	10000.0	5.000500e+03	2886.895680	1.00	2500.75	5.000500e+03	7.500250e+03	10000.00
CustomerId	10000.0	1.569094e+07	71936.186123	15565701.00	15628528.25	1.569074e+07	1.575323e+07	15815690.00
CreditScore	10000.0	6.505288e+02	96.653299	350.00	584.00	6.520000e+02	7.180000e+02	850.00
Age	10000.0	3.892180e+01	10.487806	18.00	32.00	3.700000e+01	4.400000e+01	92.00
Tenure	10000.0	5.012800e+00	2.892174	0.00	3.00	5.000000e+00	7.000000e+00	10.00
Balance	10000.0	7.648589e+04	62397.405202	0.00	0.00	9.719854e+04	1.276442e+05	250898.09
NumOfProducts	10000.0	1.530200e+00	0.581654	1.00	1.00	1.000000e+00	2.000000e+00	4.00
HasCrCard	10000.0	7.055000e-01	0.455840	0.00	0.00	1.000000e+00	1.000000e+00	1.00
IsActiveMember	10000.0	5.151000e-01	0.499797	0.00	0.00	1.000000e+00	1.000000e+00	1.00
EstimatedSalary	10000.0	1.000902e+05	57510.492818	11.58	51002.11	1.001939e+05	1.493882e+05	199992.48

```
df.describe(include=['object', 'category'])
```

	Surname	Geography	Gender	Card Type
count	10000	10000	10000	10000
unique	2932	3	2	4
top	Smith	France	Male	DIAMOND
freq	32	5014	5457	2507

```
# Class balance of the target: churners are the minority class
df["churn"].value_counts(normalize=True).round(3)
```

```
churn
0    0.796
1    0.204
Name: proportion, dtype: float64
```

```
#  SECTION 3 - Separate Features and Target
# X contains all independent features (all columns except 'churn')
X = df.drop("churn", axis=1)

# y contains the dependent variable (what we want to predict)
y = df["churn"]
```

```
#  SECTION 4 - Identify Numeric and Categorical Columns
# Remove identifier columns (the real CustomerId stays available in df for the final export)
# RowNumber is a positional index with no business meaning: it must not reach the models
cols_to_remove = ["RowNumber", "Surname", "CustomerId"]

X = X.drop(columns=[col for col in cols_to_remove if col in X.columns])

# Selects columns with object (string) dtype → categorical
categorical_cols = X.select_dtypes(include=["object"]).columns

# Selects integer or float columns → numerical
numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns
```

```
X.head().T
```

	0	1	2	3	4
CreditScore	619	608	502	699	850
Geography	France	Spain	France	France	Spain
Gender	Female	Female	Female	Female	Female
Age	42	41	42	39	43
Tenure	2	1	8	1	2
Balance	0.0	83807.86	159660.8	0.0	125510.82
NumOfProducts	1	1	3	2	1
HasCrCard	1	0	1	0	1
IsActiveMember	1	1	0	0	1
EstimatedSalary	101348.88	112542.58	113931.57	93826.63	79084.1
Complain	1	1	1	0	0
Satisfaction Score	2	3	3	5	5
Card Type	DIAMOND	DIAMOND	DIAMOND	GOLD	GOLD
Point Earned	464	456	377	350	425

```
# Make a temporary copy
temp = X.join(y).copy()

# Convert all categorical columns to numeric using pandas factorize
for col in temp.columns:
    if temp[col].dtype == "object":
        temp[col] = pd.factorize(temp[col])[0]

# Now compute correlation safely
temp.corr()["churn"].sort_values(ascending=False).head(15)
```

```
churn                1.000000
Complain             0.995693
Age                  0.285296
Geography            0.153518
Balance              0.118577
EstimatedSalary      0.012490
Point Earned        -0.004628
Satisfaction Score  -0.005849
HasCrCard           -0.006976
Card Type           -0.009484
Tenure              -0.013656
CreditScore         -0.026771
NumOfProducts       -0.047611
Gender              -0.106267
IsActiveMember      -0.156356
Name: churn, dtype: float64
```

```
df[["Complain", "churn"]].head(10)
```

	Complain	churn
0	1	1
1	1	0
2	1	1
3	0	0
4	0	0
5	1	1
6	0	0
7	1	1
8	0	0
9	0	0

Leakage check. `Complain` is almost a copy of the target (r 0.996): a customer is recorded as having complained almost exactly when they churn. The field would not be known at prediction time, so keeping it would give near-perfect and meaningless scores. The column is removed before training.

```
X = X.drop(columns=["Complain"])
```

```
X.head().T
```

	0	1	2	3	4
CreditScore	619	608	502	699	850
Geography	France	Spain	France	France	Spain
Gender	Female	Female	Female	Female	Female
Age	42	41	42	39	43
Tenure	2	1	8	1	2
Balance	0.0	83807.86	159660.8	0.0	125510.82
NumOfProducts	1	1	3	2	1
HasCrCard	1	0	1	0	1
IsActiveMember	1	1	0	0	1
EstimatedSalary	101348.88	112542.58	113931.57	93826.63	79084.1
Satisfaction Score	2	3	3	5	5
Card Type	DIAMOND	DIAMOND	DIAMOND	GOLD	GOLD
Point Earned	464	456	377	350	425

```
# Make a temporary copy
temp = X.join(y).copy()

# Convert all categorical columns to numeric using pandas factorize
for col in temp.columns:
    if temp[col].dtype == "object":
        temp[col] = pd.factorize(temp[col])[0]

# Now compute correlation safely
temp.corr()["churn"].sort_values(ascending=False).head(15)
```

```
churn                1.000000
Age                  0.285296
Geography            0.153518
Balance              0.118577
EstimatedSalary      0.012490
Point Earned        -0.004628
Satisfaction Score  -0.005849
HasCrCard           -0.006976
Card Type           -0.009484
Tenure              -0.013656
CreditScore         -0.026771
NumOfProducts       -0.047611
Gender              -0.106267
IsActiveMember      -0.156356
Name: churn, dtype: float64
```

```
y.head()
```

```
0    1
1    0
2    1
3    0
4    0
Name: churn, dtype: int64
```

```
# we need to update the numeric column because we have removed complain column
numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns
```

```
#  SECTION 5 - Preprocessing Step (ColumnTransformer)
# Preprocessing steps applied before sending data to machine learning models:
# - numeric columns → scaled (mean=0, std=1)
# - categorical columns → one-hot encoded (converted to binary format)
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_cols),                # Apply scaling to numeric data
        ("cat", OneHotEncoder(drop="first"), categorical_cols)  # Encode categoricals, dropping first level to
↪  avoid multicollinearity
    ]
)
```

```
#  SECTION 6 - Build Pipelines for Each Model
#  Logistic Regression Pipeline
# Combines preprocessing + logistic regression into a single workflow
log_reg_pipeline = Pipeline(steps=[
    ("preprocessing", preprocessor),                     # Step 1: apply all transformations
    ("model", LogisticRegression(max_iter=10000))         # Step 2: train classifier
])

#  Random Forest Pipeline
# Random Forest pipeline: same preprocessing, but a different model at the end.
rf_pipeline = Pipeline(steps=[
    ("preprocessing", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=300,          # number of trees in the forest
        max_depth=None,            # tree depth grows until pure leaves
        random_state=42            # ensures reproducibility
    ))
])
```

```
#  SECTION 7 - Split Train vs. Test Data
# Divide data into 80% training, 20% testing.
# Stratify ensures the target class distribution is preserved.
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
#  SECTION 8 - Train Models
# Fits the logistic regression pipeline to training data
log_reg_pipeline.fit(X_train, y_train)

# Fits the random forest pipeline to the training data
rf_pipeline.fit(X_train, y_train)
```

```
Pipeline(steps=[('preprocessing',
                 ColumnTransformer(transformers=[('num', StandardScaler(),
                                                  Index(['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'Has
       'IsActiveMember', 'EstimatedSalary', 'Satisfaction Score',
       'Point Earned'],
      dtype='object')),
                                                 ('cat',
                                                  OneHotEncoder(drop='first'),
                                                  Index(['Geography', 'Gender', 'Card Type'], dtype='object'))])),
                ('model',
                 RandomForestClassifier(n_estimators=300, random_state=42))])
```

1 Pipeline overview (business view)

```
# Diagram in commented code for business presentation
"""
CHURN PREDICTION PIPELINE - BUSINESS VIEW
=========================================

STEP 1: DATA PREPARATION
   ↓
• Cleaning: Missing values, outliers
• Standardization: Data scaling
• Encoding: Categorical → numerical transformation

STEP 2: MODEL TRAINING
   ↓
• LOGISTIC REGRESSION: Simple and interpretable model
• RANDOM FOREST: Powerful model for complex patterns

STEP 3: BUSINESS PREDICTIONS
   ↓
• Churn probabilities: 0% to 100%
• Customer segmentation: Low / Moderate / High / Critical risk
• Targeted actions: Personalized recommendations

"""
#3. Diagram with Business Metrics

# Create a business summary of the pipeline
def create_business_summary(pipeline, feature_names):
    business_summary = {
        "Objective": "Predict customer departure probability",
        "Input data": f"{len(feature_names)} customer variables",
        "Models used": ["Logistic Regression", "Random Forest"],
        "Business outputs": [
            "Churn risk score (0-100%)",
            "Customer segmentation by risk",
            "Personalized commercial action recommendations"
        ],
        "Benefits": [
            "Proactive detection of at-risk customers",
            "Personalization of loyalty campaigns",
            "Marketing budget optimization"
        ]
    }
    return business_summary

# Usage
business_view = create_business_summary(log_reg_pipeline, X_train.columns)
#4. Simple Graphical Visualization
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def create_business_pipeline_diagram():
    fig, ax = plt.subplots(1, 1, figsize=(12, 6))

    # Pipeline steps
    steps = [
        ("Customer Data", "Raw data\n(history, behavior)"),
        ("Cleaning", "Standardization\nCategory encoding"),
        ("Modeling", "2 algorithms:\n- Logistic Regression\n- Random Forest"),
        ("Predictions", "Churn probabilities\nRisk segmentation"),
        ("Actions", "Recommendations\ntargeted campaigns")
    ]

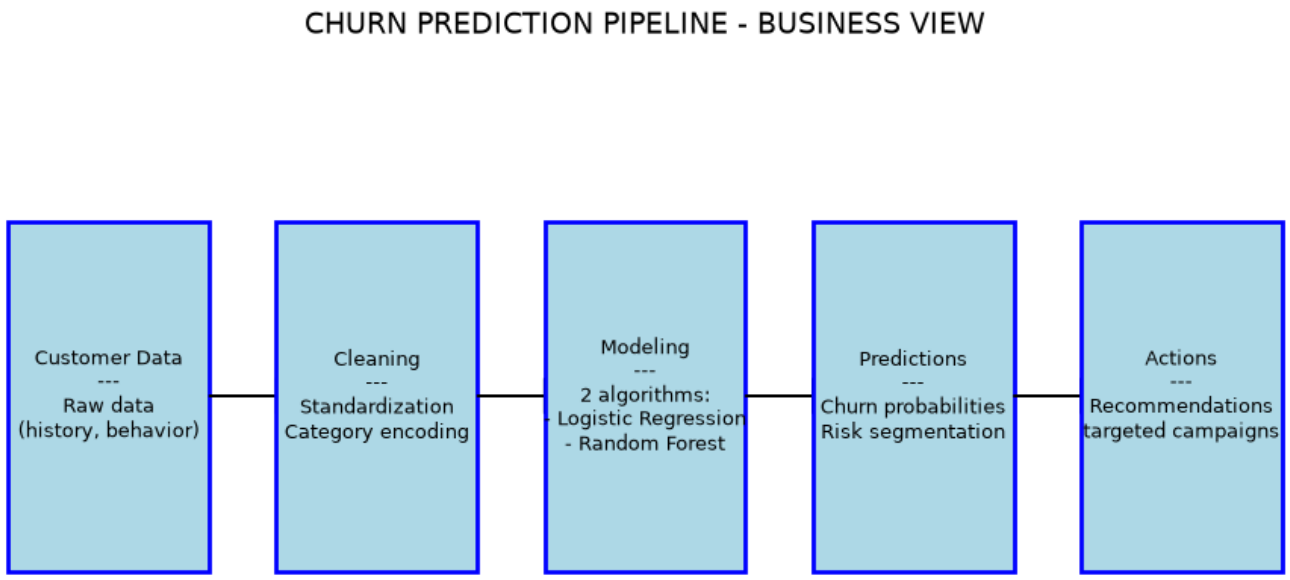
    # Draw boxes
    for i, (title, desc) in enumerate(steps):
        rect = patches.Rectangle((i*2, 0), 1.5, 1, linewidth=2,
                                 edgecolor='blue', facecolor='lightblue')
```

```
        ax.add_patch(rect)
        ax.text(i*2 + 0.75, 0.5, f"{title}\n---\n{desc}",
                ha='center', va='center', fontsize=9)

        # Arrows
        if i < len(steps)-1:
            ax.arrow(i*2 + 1.5, 0.5, 0.5, 0, head_width=0.1, head_length=0.1, fc='blue')

    ax.set_xlim(-0.5, len(steps)*2)
    ax.set_ylim(-0.5, 1.5)
    ax.set_title("CHURN PREDICTION PIPELINE - BUSINESS VIEW", fontsize=14)
    ax.axis('off')
    plt.show()

create_business_pipeline_diagram()
```




```
#  SECTION 9 - Predictions
# Produces predicted class labels (0 or 1)
log_reg_pred = log_reg_pipeline.predict(X_test)
rf_pred = rf_pipeline.predict(X_test)
```

```
#  SECTION 10 - Evaluate Each Model
from sklearn.metrics import roc_auc_score

#  Logistic Regression
print("  Logistic Regression Results")
print("Accuracy:", accuracy_score(y_test, log_reg_pred))  # proportion of correct predictions
print(confusion_matrix(y_test, log_reg_pred))             # matrix of TP, FP, FN, TN
print(classification_report(y_test, log_reg_pred))        # precision, recall, F1 score
print("ROC-AUC:", round(roc_auc_score(y_test, log_reg_pipeline.predict_proba(X_test)[:, 1]), 3))

#  Random Forest
print("  Random Forest Results")
print("Accuracy:", accuracy_score(y_test, rf_pred))
print(confusion_matrix(y_test, rf_pred))
print(classification_report(y_test, rf_pred))

print("ROC-AUC:", round(roc_auc_score(y_test, rf_pipeline.predict_proba(X_test)[:, 1]), 3))
```

```
  Logistic Regression Results
Accuracy: 0.813
```

```
[[1540   52]
 [ 322   86]]
              precision    recall  f1-score   support

           0       0.83      0.97      0.89      1592
           1       0.62      0.21      0.32       408

    accuracy                           0.81      2000
   macro avg       0.73      0.59      0.60      2000
weighted avg       0.79      0.81      0.77      2000

ROC-AUC: 0.778
 Random Forest Results
Accuracy: 0.8685
[[1544   48]
 [ 215  193]]
              precision    recall  f1-score   support

           0       0.88      0.97      0.92      1592
           1       0.80      0.47      0.59       408

    accuracy                           0.87      2000
   macro avg       0.84      0.72      0.76      2000
weighted avg       0.86      0.87      0.85      2000

ROC-AUC: 0.867
```

```python
# Business reading of the selected model: every figure is computed from the test set
from sklearn.metrics import precision_score, recall_score

tn, fp, fn, tp = confusion_matrix(y_test, rf_pred).ravel()
business_metrics = {
    "Accuracy":                   f"{accuracy_score(y_test, rf_pred):.1%}",
    "Precision on flagged":       f"{precision_score(y_test, rf_pred):.1%}",
    "Churners detected (recall)": f"{recall_score(y_test, rf_pred):.1%}",
    "False-positive rate":        f"{fp / (tn + fp):.1%}",
    "Churners missed":            f"{fn} of {fn + tp}",
}

print("RANDOM FOREST - TEST SET SUMMARY:")
for metric, value in business_metrics.items():
    print(f"• {metric}: {value}")
```

```
RANDOM FOREST - TEST SET SUMMARY:
• Accuracy: 86.9%
• Precision on flagged: 80.1%
• Churners detected (recall): 47.3%
• False-positive rate: 3.0%
• Churners missed: 215 of 408
```

2 Understanding the results

The test set holds 2,000 customers, of whom 408 churned. The churn base rate is 20.4%, so a model that always answers “no churn” would already reach 79.6% accuracy. Accuracy alone therefore says little: the model must be judged on the churn class.

2.1 Logistic Regression

Accuracy 0.813, confusion matrix `[[1540 52] [322 86]]`.

- Very good on class 0 (non-churn), but only because it mostly predicts “no churn”.
- Recall on churn = 0.21: it finds 86 of the 408 churners. F1 = 0.32.
- Its accuracy is barely above the 0.796 trivial baseline.

2.2 Random Forest

Accuracy 0.869, confusion matrix `[[1544 48] [215 193]]`.

- Precision on churn = 0.80: when the model flags a customer, it is right about 4 times out of 5.
- Recall on churn = 0.47: it finds 193 of the 408 churners and still misses 215.
- F1 = 0.59, false-positive rate = 3.0% (48 of 1,592 non-churners flagged).

2.3 Which model is better?

Metric	Logistic Regression	Random Forest
Accuracy	0.813	0.869
Precision (churn)	0.62	0.80
Recall (churn)	0.21	0.47
F1-score (churn)	0.32	0.59
ROC-AUC	0.778	0.867

Random Forest is the better choice: it more than doubles recall on the churn class while also raising precision. It is a prioritisation tool rather than an exhaustive early-warning system, because about half of the churners are still missed.

2.4 Why is churn hard to predict here?

Once the leakage column `Complain` is removed, the remaining linear signals are weak. The strongest correlations with the target are Age (+0.285), IsActiveMember (-0.156), Geography (+0.154) and Balance (+0.119); salary, card type, credit card ownership, tenure and satisfaction score are close to zero. Part of the signal is non-linear (for example the number of products), which Random Forest captures and Logistic Regression does not.

```
#  SECTION 11 - Compare Models Automatically
log_acc = accuracy_score(y_test, log_reg_pred)
rf_acc = accuracy_score(y_test, rf_pred)

print("  Model Comparison")
print(f"Logistic Regression Accuracy: {log_acc:.4f}")
print(f"Random Forest Accuracy:       {rf_acc:.4f}")

best_model = "Random Forest" if rf_acc > log_acc else "Logistic Regression"
print("\n  Best Model:", best_model)
```

```
  Model Comparison
Logistic Regression Accuracy: 0.8130
Random Forest Accuracy:       0.8685

  Best Model: Random Forest
```

```
# Which features drive the Random Forest? (impurity-based importance, top 10)
feature_names = rf_pipeline.named_steps["preprocessing"].get_feature_names_out()
importances = pd.Series(rf_pipeline.named_steps["model"].feature_importances_, index=feature_names)
importances.sort_values(ascending=False).head(10).round(3)
```

```
num__Age                  0.211
num__NumOfProducts        0.123
num__Balance              0.109
num__EstimatedSalary      0.105
num__CreditScore          0.104
num__Point Earned         0.102
num__Tenure               0.061
num__Satisfaction Score   0.041
num__IsActiveMember       0.036
cat__Geography_Germany    0.027
dtype: float64
```

3 1. Individual probabilities (customer-level view)

```
# Predict probabilities for the test set
y_proba = rf_pipeline.predict_proba(X_test)[:, 1]

# Keep the real customer identifier (it was dropped from the features, not from df)
customer_ids = df.loc[X_test.index, "CustomerId"]

# Create DataFrame for business readability
customer_scores = pd.DataFrame({
    "CustomerId": customer_ids.values,
    "Churn_Probability": y_proba
})

# Sort by highest risk
customer_scores = customer_scores.sort_values("Churn_Probability", ascending=False)
customer_scores.head(10)
```

	CustomerId	Churn_Probability
766	15647725	0.943333
780	15634551	0.940000
1402	15589017	0.930000
1139	15585362	0.930000
1494	15786196	0.920000
406	15696989	0.920000
614	15806808	0.916667
1169	15703793	0.913333
1938	15813113	0.913333
1047	15779176	0.906667

4 2. Segment probabilities into risk bands

```
# Single reference definition of the risk bands (used everywhere below)
RISK_BANDS = [(0.20, "Low risk (0-20%)"),
              (0.40, "Moderate (20-40%)"),
              (0.70, "High risk (40-70%)"),
              (1.01, "Critical (70-100%)")]

def risk_level(p):
    for cut, label in RISK_BANDS:
        if p < cut:
            return label

customer_scores["Risk_Level"] = customer_scores["Churn_Probability"].apply(risk_level)

customer_scores.head(1500)
```

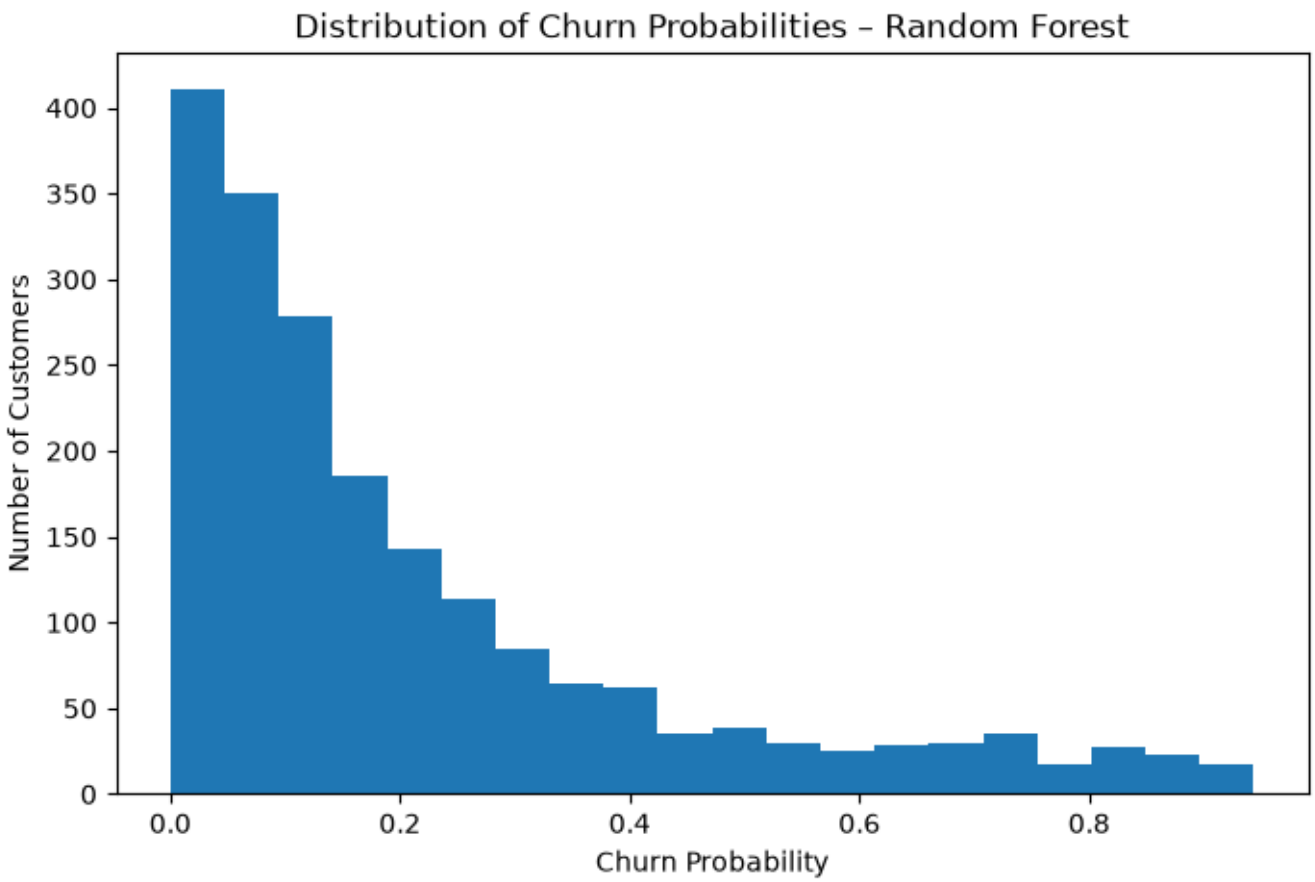
	CustomerId	Churn_Probability	Risk_Level
766	15647725	0.943333	Critical (70-100%)
780	15634551	0.940000	Critical (70-100%)
1402	15589017	0.930000	Critical (70-100%)
1139	15585362	0.930000	Critical (70-100%)
1494	15786196	0.920000	Critical (70-100%)
...	...	...	...
1755	15624424	0.060000	Low risk (0-20%)
1637	15589437	0.060000	Low risk (0-20%)
762	15660385	0.060000	Low risk (0-20%)
1387	15664734	0.060000	Low risk (0-20%)
1436	15679743	0.060000	Low risk (0-20%)

5 3. Distribution of the predicted probabilities

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.hist(y_proba, bins=20)
```

```
plt.title("Distribution of Churn Probabilities - Random Forest")
plt.xlabel("Churn Probability")
plt.ylabel("Number of Customers")
plt.show()
```



6 4. Example customers by probability level

```
# A few example customers across the probability range

examples = customer_scores.sample(5, random_state=42)
examples
```

	CustomerId	Churn_Probability	Risk_Level
82	15692583	0.016667	Low risk (0-20%)
689	15641250	0.390000	Moderate (20-40%)
904	15735522	0.080000	Low risk (0-20%)
1594	15677052	0.153333	Low risk (0-20%)
184	15598892	0.086667	Low risk (0-20%)

7 5. Share of customers in each risk band

```
# Share of scored customers in each risk band (%)
risk_summary = (customer_scores["Risk_Level"].value_counts(normalize=True) * 100).round(1)
risk_summary
```

```
Risk_Level
Low risk (0-20%)      62.4
Moderate (20-40%)     20.2
```

```
High risk (40-70%)    11.1
Critical (70-100%)     6.2
Name: proportion, dtype: float64
```

8 6. Export the customer churn probabilities (CSV / Excel)

```
#Step 1 - Build an exportable table (real CustomerId + probability + risk band)
customer_export = customer_scores[["CustomerId", "Churn_Probability", "Risk_Level"]].copy()

#Step 2 - Export to CSV
customer_export.to_csv("customer_churn_predictions.csv", index=False)
print("CSV file saved: customer_churn_predictions.csv")

#Step 3 - Export to Excel
customer_export.to_excel("customer_churn_predictions.xlsx", index=False)
print("Excel file saved: customer_churn_predictions.xlsx")
print(f"{len(customer_export)} customers exported (test set only)")
```

```
CSV file saved: customer_churn_predictions.csv
Excel file saved: customer_churn_predictions.xlsx
2000 customers exported (test set only)
```

9 Scoring new customers with the saved Random Forest pipeline

`rf_pipeline` contains the preprocessing (scaling, one-hot encoding) and the Random Forest model, so new customer data can be fed to it directly.

1. Load the customers to score. The file must contain the same feature columns as the training data (same names and types): `CreditScore`, `Geography`, `Gender`, `Age`, `Tenure`, `Balance`, `NumOfProducts`, `HasCrCard`, `IsActiveMember`, `EstimatedSalary`, `Satisfaction Score`, `Card Type`, `Point Earned`.

```
new_customers = pd.read_csv("NEW_CUSTOMERS_DATA.csv")
```

2. Predict the churn probability.

```
new_customers["Churn_Probability"] = rf_pipeline.predict_proba(new_customers)[:, 1]
```

3. Apply the risk bands. Use the single `risk_level()` function defined above, so that every output shares the same cut-offs (0.20 / 0.40 / 0.70).

```
new_customers["Risk_Level"] = new_customers["Churn_Probability"].apply(risk_level)
```

4. Export for Power BI.

```
new_customers.to_csv("CUSTOMERS_WITH_CHURN_PROBABILITIES.csv", index=False)
```

> **Note: these probabilities are uncalibrated.** They rank customers reliably from least to most likely to churn, but the absolute value should not be read as a literal percentage chance. Wrapping the classifier in `CalibratedClassifierCV` (isotonic, 5 folds) would be the next step if the absolute values need to be meaningful.

10 Save the selected model (Random Forest pipeline)

```
import joblib, sklearn

# The selected model is the Random Forest pipeline (preprocessing + classifier)
joblib.dump(rf_pipeline, "churn_model.pkl", compress=3)
print("Random Forest pipeline saved to churn_model.pkl")
print("scikit-learn version:", sklearn.__version__)
```

```
Random Forest pipeline saved to churn_model.pkl
scikit-learn version: 1.6.1
```

11 Conclusion

This project shows how a supervised model can rank bank customers by churn risk and feed a decision-support layer.

Leakage removal. The `Complain` feature was almost perfectly correlated with the target (r 0.996) and acted as a leakage variable. It was removed, together with the identifier columns (`RowNumber`, `CustomerId`, `Surname`). This is the decision that makes the reported performance realistic.

Model selection. On the stratified hold-out set (2,000 customers, 408 churners), the Random Forest reaches an accuracy of 0.869, a precision of 0.80 and a recall of 0.47 on the churn class (ROC-AUC 0.867). Logistic Regression reaches 0.813 accuracy but only 0.21 recall. The Random Forest pipeline is the one saved to `churn_model.pkl` and served by the Streamlit application.

What the score supports. The model is right about 4 times out of 5 when it flags a customer, and it finds roughly half of the customers who leave. It is a prioritisation tool for retention teams: 6.2% of scored customers fall in the Critical band and 11.1% in the High band.

Limitations. The probabilities are not calibrated, so they are a ranking rather than a literal chance of leaving. No hyperparameter tuning or class rebalancing was applied; both are realistic levers for raising recall. Scores were produced for the 2,000 test customers only.

Next steps. Probability calibration, hyperparameter search and class rebalancing, scoring of the full customer base, and model monitoring.